

설비 고장을 예측하는 AI 데이터 분석

환경 설정 및 **Python** 시작

설비 데이터와 Python 환경

설비 고장을 예측하는 AI 데이터 분석

환경 설정 및 Python 시작

설비 데이터와 Python 환경

Part 1

예측 정비와 설비 데이터

Python을 쓰는 이유

개발 환경 3가지

Python 첫 코드와 오류

Part 2

코드 실행과 print()

설비 데이터 출력

오류 메시지 읽기

설비 데이터와 Python 환경

고장을 미리 읽는 예측 정비

- 설비 데이터를 지켜보다 고장 신호를 미리 잡아 대응하는 방식
- 판단의 근거는 진동·온도·압력 같은 데이터
- 데이터로 미리 관리하는 것은 설비의 건강검진

정비 방식 세 가지 비교

사후 정비

고장 후 수리 → 이미 멈춘 다음이기 때문에 손해가 큼

예방 정비

주기마다 점검 → 멀쩡한 부품도 버리고, 주기 사이 고장은 못 막음

예측 정비

데이터로 미리 신호를 잡아 정비를 계획 → 우리가 배울 방식

예측 정비가 만드는 가치

비계획 정지 감소

설비 수명 연장

현장 안전 향상

데이터 인재와 진로 연결

이 가치를 만들려면 누군가는 설비 데이터를 읽고 해석해야 함

과거에는 숙련 기술자의 감에 의존했지만, 이제는 데이터로 누구나 객관적으로 확인 가능

누구나 기본기와 꾸준함으로 **스마트 제조 데이터 직무**의 출발점에 설 수 있음

데이터 직무가 하는 세 가지 일

수집·정리

데이터 다듬기 → 빈 칸·튀 값 분석 가능하게 정리

분석·시각화

그래프로 요약 → 숫자 수천 개를 그림 하나로

판단 전달

점검 문장화 → "3번 설비 점검 필요"처럼 행동할 문장으로

모든 일의 도구는 Python

- 세 가지 일 모두 데이터를 다루는 도구에서 출발
- 그 도구가 바로 우리가 배울 Python
- 그래서 첫 단원에서 Python을 탄탄히 다지기

자주 보는 설비 데이터 예시

항목	무엇을 나타냄	이상 신호
온도	뜨거운 정도 (무리하면 열↑)	계속 오름 (71→82°C)
진동	떨림 (부품 닳거나 헐거워짐)	떨림 폭 커짐 (2.1→4.0)
압력	미는 세기 (새거나 막힘)	갑자기 뚝 떨어짐/치솟음

설비 데이터는 모두 숫자

- 온도 75, 진동 2.3, 압력 4.1처럼 전부 숫자
- 숫자라서 계산하고 그래프로 그릴 수 있음
- 평소와 비교할 수 있어 데이터 분석이 가능

센서가 데이터를 만드는 과정

설비 가동



센서 측정



기록



시간순 누적

데이터는 시간 순서로 쌓임

- 데이터는 가로에 항목, 세로에 시간 순서 측정값
- 한 시점만 보면 정상 여부 판단 어려움
- 시간 흐름을 봐야 점점 나빠지는 변화 포착

정상과 이상의 차이

정상

평소 범위 안 → 늘 70도 근처였다면 그게 정상

이상

범위에서 벗어남 → 핵심은 절대 숫자가 아니라 평소와의 차이 (체온 36.5 vs 39도)

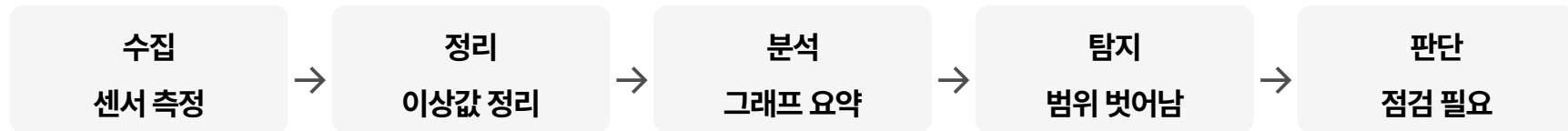
이상 징후의 두 가지 모양

이상 징후는 두 가지 모양으로 나타남

1. 급변형 - 값이 갑자기 확 튀는 경우
2. 완만형 - 천천히 조금씩 변해가는 경우

완만형은 **그래프로 흐름**을 봐야 비로소 보임

데이터에서 정비 판단까지 5단계



5단계는 과정 전체의 지도

- 이 다섯 단계(수집→정리→분석→탐지→판단)가 교육과정 전체의 지도
- 모든 단계에서 데이터를 만지는 도구가 Python
- 그래서 지금 그 도구부터 손에 익히기

측정값 읽어보기

- 코딩 없이 눈으로 데이터를 읽는 워밍업
- 정답 맞추기가 아니라 이상이 보인다는 경험이 목적
- 정상 표와 이상 표를 비교해 다른 값 찾기

정상 측정값 표

시각	온도	진동	압력
09:00	71	2.1	4.0
09:01	70	2.2	4.1
09:02	72	2.0	4.0
09:03	71	2.1	4.0

이상 징후 측정값 표

시각	온도	진동	압력
10:01	74	2.4	4.0
10:02	77	3.1	3.9
10:03	82	4.0	3.7

찾기와 점검

진행 순서

1. 정상 표로 평소 범위 함께 확인
2. 이상 표에서 다른 줄 각자 찾기
3. 2~3명씩 찾은 것과 이유 나누기

점검 - 온도와 진동이 함께 **계속 상승**한다는 흐름을 발견

Python을 쓰는 세 가지 이유

읽기 쉬움

영어처럼 읽힘 → 처음 하는 사람도 빠르게 익숙해짐

무료

누구나 사용 → 공짜(오픈소스)에 자료가 많아 검색으로 해결

도구 풍부

분석에 강함 → 큰 데이터·그래프·머신러닝까지 하나로

배우기 쉽고 데이터에 강함

- Python은 배우기 쉽고, 공짜이고, 데이터에 강함

- 그래서 설비 데이터 분석의 표준 도구

- 현장에서 실제로 쓰이는 도구를 배우는 것

Python으로 하게 될 일

불러오기
데이터 열기



정리
빈칸 채우기



시각화
그래프 요약



예측
고장 예측

실제 설비 데이터로 연습

연습에 쓰는 실제 데이터

1. 항공기 엔진 작동을 시뮬레이션한 데이터
2. 산업용 기계 소리에서 뽑아낸 데이터

화려한 머신러닝도 **데이터를 불러와 출력하는** 기본 위에 쌓임

코드 작성과 실행의 의미

코드 작성
명령 적기



실행
버튼 누르기



결과 확인
화면에 결과

실습 환경 세 가지

Colab

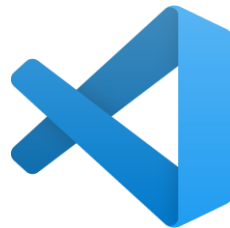
크롬만 열면 시작, 코드 실행을 구글 컴퓨터가 담당

Jupyter

내 컴퓨터에서 코드 실행 (코랩과 화면·사용법이 닮음)

VSCode

내 컴퓨터가 코드를 실행하는 전문 도구



코랩으로 시작하는 이유

- 세 개를 동시에 완벽히 다룰 필요 없음
- 가장 쉬운 코랩으로 시작해 즐거움 먼저
- 그다음 VSCode로 자연스럽게 확장

설치 없는 환경 구글 코랩

- 브라우저에서 설치 없이 바로 코드 실행
- 구글 계정만 있으면 즉시 시작
- 수업 실습은 대부분 코랩에서 진행

클라우드와 셀 단위 작업

코랩이 편한 이유

1. 코드를 돌리는 컴퓨터가 **인터넷 너머 구글 컴퓨터** (클라우드)
2. 내 기기 성능과 상관없이 똑같이 작동

작업은 **셀**이라는 칸 단위로, 한 토막 쓰고 실행하면 바로 아래 결과 표시

주피터 노트북과 코랩의 관계

- 주피터는 코드와 결과를 셀 단위로 정리하는 환경
- 코랩은 주피터를 인터넷에서 쓰게 만든 것
- 둘은 모양과 사용법이 거의 같음

내 컴퓨터용 도구 VSCode

- 마이크로소프트가 만든 무료 코드 편집기
- 내 컴퓨터에서 코드를 파일로 저장하며 실행
- 현장에서 가장 많이 쓰는 전문 도구

VSCode 사용 준비물

처음 쓰려면 세 가지 준비 필요

1. VSCode 프로그램 설치
2. Python 따로 설치
3. Python을 다룰 **확장** 추가

준비는 뒤에서 한 단계씩 직접 직접 하며 진행

세 환경 한눈에 비교

환경	설치	실행 위치
Colab	불필요	구글 컴퓨터
Jupyter	필요	내 컴퓨터
VSCoDe	필요	내 컴퓨터

입문은 코랩 본격은 VSCode

- 입문은 코랩, 본격 작업은 VSCode
- 실제 분석가도 코랩으로 탐색 후 VSCode로 이동
- 주피터는 코랩과 닮아 개념만 알아두기

클라우드 실행과 내 컴퓨터 실행

코랩

인터넷 너머 실행

VSCode

내 컴퓨터 실행

셀 실행과 파일 실행

셀 실행

한 토막씩 → 데이터를 한 단계씩 확인 (요리를 단계마다 맛보기)

파일 실행

통째로 한 번에 → 파일 전체를 위에서 아래로 (레시피 전체를 한 번에)

코랩 화면 한눈에 익히기

메뉴 막대

위쪽 파일·수정·런타임 등 → 실행/저장/연결이 여기

셀

코드나 글을 적는 칸 → 코드 셀 / 텍스트 셀 두 종류

실행 표시

셀 왼쪽 동그라미 → 돌면 실행 중, 숫자면 완료

VSCode 화면 한눈에 익히기

편집 영역 가운데 넓은 곳 → 코드를 쓰는 곳

탐색기 왼쪽 → 내 폴더와 파일 목록 (Ctrl+Shift+E)

터미널 아래 검은 칸 → 실행 결과가 나오는 곳 (Ctrl+`)

노트북(.ipynb)과 파이썬 파일(.py)

.ipynb

셀 단위로 나눠 실행 → 코랩·주피터가 쓰는 노트북 형식

.py

위에서 아래로 한 번에 실행 → VSCode가 쓰는 순수 코드 파일

차이점

실행 단위(셀 vs 파일 전체)와 결과 위치가 다름

내 코드는 어디에 저장될까

폴더

파일을 담는 상자. 실습용 폴더를 하나 만들어 두면 정리가 쉬움

파일

코드가 저장된 하나의 문서 (예: first.py)

경로

파일의 주소 (예: 바탕화면 → 파이썬연습 → first.py)

코랩 노트북 만들기

- 목표 - 코랩에 접속해 새 노트북 만들기

- 첫 실습이니 한 단계씩 함께 진행

- 안 보이는 게 있으면 바로 손 들기

코랩 접속 단계

진행 순서

1. 브라우저 열기 (크롬 권장)
2. Colab 검색해 사이트 들어가기
3. 구글 계정으로 로그인
4. 새 노트 버튼으로 빈 노트북 만들기
5. 파일 이름을 **내 첫 노트북**으로 바꾸기

코랩 점검

점검

1. 코랩 화면이 열렸는가
2. 새 노트북이 만들어졌는가
3. 파일 이름을 바꿔봤는가

상황 - 화면이 영어여도 버튼 위치는 같으니 그대로 진행

첫 코드 셀 실행

- 목표 - 첫 코드를 입력하고 실행해 결과 확인

- 결과 한 줄이 뜨면 그게 첫 코딩 성공

- `print`의 자세한 문법은 다음 모듈에서

print 입력과 실행

빈 셀에 입력 후 실행

CODE

```
print("내 첫 코드 실행 성공")
```

실행 버튼(▶) 또는 Shift+Enter, 글자를 내 이름으로 바꿔 재실행

▶ 실행하면 화면에 이렇게 나와요

내 첫 코드 실행 성공

첫 실행 점검

점검

1. 코드를 입력했는가
2. 실행 버튼이나 Shift(or Ctrl)+Enter로 실행했는가
3. 셀 아래 결과가 나타났는가

상황 - **빨간 글씨**가 떠도 잘못이 아니라 자연스러운 과정

셀 다루기

- 목표 - 셀 추가·삭제·이동과 메모 익히기
- 셀을 다루면 노트북을 내 것처럼 사용 가능
- 직접 해보기

셀 추가와 메모

진행 순서

1. 코드 셀 추가 후 `print(1 + 1)` 실행해 2 확인
2. 텍스트 셀 추가해 **연습 노트** 메모 적기
3. 셀을 위아래로 이동
4. 필요 없는 셀 삭제

셀 조작 점검

점검

1. 코드 셀을 추가하고 실행했는가
2. 텍스트 셀로 메모를 남겼는가
3. 셀을 이동하고 삭제했는가

꼭 외우는 필수 단축키

1. Shift+Enter : 셀 실행 후 다음 셀로 (가장 많이 씀)
2. Ctrl+Enter : 셀 실행하고 그 자리에 머무르기
3. Ctrl+M+N / Ctrl+M+P : 위/아래로 셀 이동 * 코랩·주피터
4. Ctrl+M+A / Ctrl+M+B : 위(Above) / 아래(Below)에 새 셀 추가 * 코랩·주피터
5. Ctrl+M+M : 현재 셀 텍스트셀로 변경 * 코랩·주피터
6. Ctrl+M+D : 선택한 셀 삭제 * 코랩·주피터

코랩 연결이 끊겼을 때

진행 순서

1. 화면 오른쪽 위 '연결' 또는 '다시 연결' 버튼 누르기
2. 초록 체크(RAM·디스크 표시)가 뜨면 다시 연결된 것
3. 만들어둔 값이 사라졌으므로 맨 위 셀부터 순서대로 다시 실행
4. 빠르게 하려면 '런타임 → 모두 실행'으로 한 번에 실행

코드가 너무 무거워 멈출 때

진행 순서

1. 멈춘 것 같으면 먼저 '런타임 → 실행 중단'으로 지금 셀을 멈추기
2. 무한 반복이 의심되면 반복 횟수를 작게 줄여 다시 실행 (예: 백만 → 천)
3. 'RAM 부족' 경고가 뜨면 '런타임 → 런타임 다시 시작' 후 데이터를 나눠서 처리
4. 그래도 안 되면 print로 어디까지 실행됐는지 확인해 무거운 지점 찾기

VSCode 설치

- 목표 - VSCode를 내 컴퓨터에 설치하기

- 설치하는 프로그램 하나 까는 것과 같음

- 윈도우와 맥 화면을 함께 안내

VSCode 내려받기

진행 순서

1. VSCode 다운로드 검색
2. **공식 사이트** code.visualstudio.com 들어가기
3. 내 컴퓨터에 맞는 버튼 누르기
4. 설치 파일 실행 후 동의·다음으로 설치
5. VSCode 실행해 화면 확인

설치 점검

점검

1. 공식 사이트에서 내려받았는가
2. 설치가 정상적으로 끝났는가
3. VSCode가 실행되어 화면이 열렸는가

상황 - 설치하는 시간이 걸릴 수 있으니 멈춘 듯해도 **잠시 대기**

Python 준비

- 목표 - Python 설치와 확장 추가하기
- VSCode는 빈 작업실, Python과 부품이 필요
- 직접 실습 중심으로 진행

확장과 인터프리터

진행 순서

1. python.org에서 Python 설치 (윈도우는 **Add to PATH** 체크)
2. VSCode 확장에서 Python 검색해 설치
3. 사용할 Python 버전 (인터프리터 = 코드를 실행해 주는 프로그램) 선택

준비 점검

점검

1. Python을 설치했는가
2. Python 확장을 설치했는가
3. 사용할 버전을 선택했는가

상황 - Add to PATH를 놓쳤으면 **재설치**하며 체크

VSCode 확장 설치 자세히 보기

진행 순서

1. 왼쪽 세로 막대에서 네모 4개 아이콘(Extensions) 클릭 ※ 단축키 Ctrl+Shift+X
2. 검색창에 'Python' 입력
3. 파란 배지의 Microsoft 공식 확장 선택
4. [Install] 버튼 클릭 후 잠시 대기

저장하면 코드가 저절로 정리되는 설정

진행 순서

1. 확장에서 'Black Formatter'(Microsoft) 설치
2. 파일 → 기본 설정 → 설정(Settings) 열기 ※ Ctrl+,
3. 검색창에 'format on save' 입력 후 체크박스 켜기
4. 'default formatter'를 검색해 Black Formatter로 지정

코드를 실행하는 세 가지 방식

① 웹 코랩

브라우저에서 바로. 설치 0단계. 구글 컴퓨터가 실행

② VSCode(로컬)

내 컴퓨터에 설치. 내 컴퓨터가 실행. 파일로 저장

③ VSCode+코랩 엔진

화면은 VSCode, 실행 힘은 코랩에서 빌림

VSCode에서 코랩 엔진 연결하기

진행 순서

1. VSCode 확장(Ctrl+Shift+X)에서 'Google Colab' 검색해 설치
2. 노트북 파일(.ipynb) 새로 만들거나 열기
3. 오른쪽 위 'Select Kernel'(커널 선택) 클릭
4. 'Colab' → 'Auto Connect' 선택 후 구글 로그인

VSCode+코랩 자주 쓰는 명령

드라이브 연결

'Colab: Mount Google Drive to Server' — 구글 드라이브 파일 불러오기

서버 정리

'Colab: Remove Server' — 쓰던 코랩 서버 정리

로그아웃

'Colab: Sign Out' — 코랩 계정 로그아웃

실습 - 세 방식으로 같은 코드 실행

같은 코드를 세 방식에 각각 넣고 결과가 똑같은지 확인 (환경 확인용, 문법은 다음 모듈)

CODE

```
print("온도 측정값")  
print(75)
```

▶ 실행하면 화면에 이렇게 나와요

```
온도 측정값  
75
```

코딩을 돕는 AI 기능

코랩 Gemini 코랩에 내장된 AI. 코드 설명·생성·오류 도움을 요청

자동완성 코드를 치면 다음에 올 코드를 회색으로 미리 제안 (Tab 수락)

오류 도움 오류 옆 'AI에게 설명 요청' 버튼으로 바로 질문

우리가 다룰 설비 데이터 구경하기

항공기 엔진 데이터 엔진 작동을 기록한 센서값 — 남은 수명 예측에 쓰임

기계 소리 데이터 산업 기계 소리에서 뽑은 값 — 이상 소음 탐지에 쓰임

공통점 둘 다 시간에 따라 쌓인 숫자 표. 온도·진동과 같은 구조

설비 데이터 어디서 찾을까

1. Kaggle(kaggle.com) 검색창에 데이터 이름 입력 — 무료 회원가입 후 바로 내려받기
2. 항공기 엔진 → 'NASA C-MAPSS' 또는 'turbofan degradation' 검색
3. 기계 소리 → 'MIMII' 검색 (밸브·펌프·팬 정상/이상 소리)
4. 데이터 페이지의 'Data' 탭에서 미리보기 표로 생김새 먼저 구경

해외 설비 데이터 찾는 곳

Kaggle

kaggle.com — 검색·미리보기가 가장 쉬움. 무료 가입 후 바로 다운로드

UCI ML 저장소

archive.ics.uci.edu — 예측정비 표준 데이터(AI4I 2020, MetroPT 등)

NASA · Zenodo

NASA PCoE(엔진 C-MAPSS) / Zenodo(기계 소리 MIMII) — 원본 출처

국내 설비 데이터 찾는 곳

AI-Hub

aihub.or.kr — 정부 구축 AI 학습데이터. '제조현장 이송장치 열화 예지보전' 등 설비 데이터 다수

공공데이터포털

data.go.kr — 정부·공공기관 개방 데이터. 제조·에너지·설비 관련 다양

주의

AI-Hub는 다운로드가 PC에서만 가능하고, 일부는 이용 신청·승인이 필요

데이터 분석 도구 이름 익히기

pandas

표 데이터를 다루는 도구 → 엑셀 같은 표를 코드로 처리

numpy

숫자 계산을 빠르게 하는 도구 → 대량의 숫자 연산 담당

matplotlib

그래프를 그리는 도구 → 숫자를 그림으로 요약

감사합니다.